Learning Objectives

1. What is A* Search?
2. Implement A* Search

1. A* Search

The A* Search algorithm is one of the best and popular technique used in path-finding and graph traversals.

The A* Search algorithms, unlike other traversal techniques uses intelligence. And it is also worth mentioning that many games and web-based maps use this algorithm to find the shortest path very efficiently (approximation).

Consider a square grid having many obstacles and we are given a starting cell and a target cell. We want to reach the target cell (if possible) from the starting cell as quickly as possible. What A* Search Algorithm does is that at each step it picks the node according to a value-'**f**' which is a parameter equal to the sum of two other parameters – '**g**' and '**h**'. At each step it picks the node/cell having the lowest 'f', and process that node/cell. We define 'g' and 'h' as below:

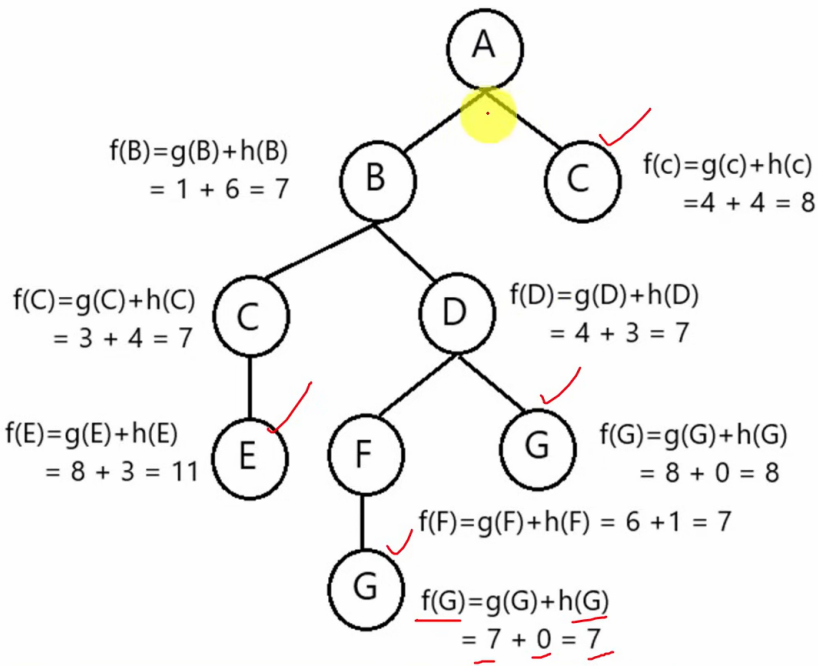
g = the movement cost to move from the starting point (initial state) to a given square (current node) on the grid (graph), following the path generated to get there.

h = the estimated movement cost to move from that given square on the grid to the final destination. This is often referred to as the heuristic, which is nothing but a kind of smart guess.

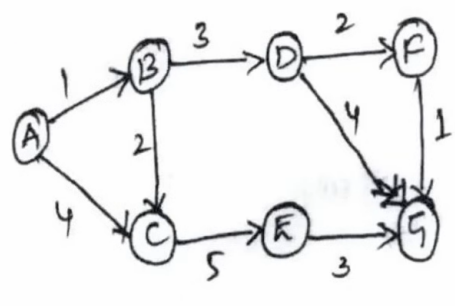
➔ *Estimated cost from the state at **node n** to a goal state*

A* Search

Now let us try to implement a basic A* search to generate a traversal path from the graph shown in Figure below



f(n) = g(n) + h(n)



A	5
B	6
C	4
D	3
E	3
F	1
G	0

Path: A→B→D→F→G
Path Cost: 7

Now based on the understanding developed above let us try to create a code to find route from A to J using A* search.

```
graph = {
    'A': {'B': 1, 'C': 4},
    'B': {'D': 3, 'C': 2},
    'C': {'E': 5},
    'D': {'F': 2, 'G': 4},
    'E': {'G': 3},
    'F': {'G': 1},
    'G': {}
}

h = {
    'A': 5,
    'B': 6,
    'C': 4,
    'D': 3,
    'E': 3,
    'F': 1,
    'G': 0
}
```

```python
def A_star_search(start, goal):
    open_list = [start]
    g_cost = {start: 0}
    parent = {start: None}

    while open_list:
        current = min(open_list, key=lambda x: g_cost[x] + h[x])

        if current == goal:
            path = []
            while current is not None:
                path.append(current)
                current = parent[current]
            path.reverse()
            print("Path found:", " → ".join(path))
            print("Total cost:", g_cost[goal])
            return

        open_list.remove(current)

        for neighbor, cost in graph[current].items():
            temp_g = g_cost[current] + cost

            if neighbor not in g_cost or temp_g < g_cost[neighbor]:
                g_cost[neighbor] = temp_g
                parent[neighbor] = current
                if neighbor not in open_list:
                    open_list.append(neighbor)

    print("No path found!")

A_star_search('A', 'G')
```

Output

```
Path found: A → B → D → F → G
Total cost: 7
```

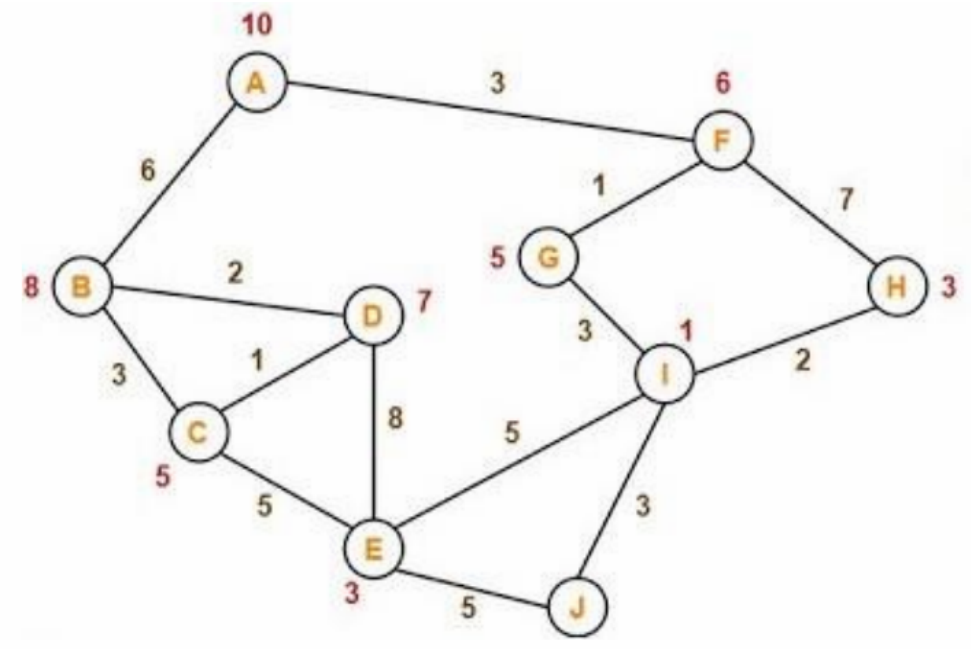
Code:

```
graph = {
    'A': {'B': 1, 'C': 4},
    'B': {'D': 3, 'C': 2},
    'C': {'E': 5},
    'D': {'F': 2, 'G': 4},
    'E': {'G': 3},
    'F': {'G': 1},
    'G': {}
```

```
}

h = {
    'A': 5,
    'B': 6,
    'C': 4,
    'D': 3,
    'E': 3,
    'F': 1,
    'G': 0
}

def A_star_search(start, goal):
    open_list = [start]
    g_cost = {start: 0}
    parent = {start: None}

    while open_list:
        current = min(open_list, key=lambda x: g_cost[x] + h[x])

        if current == goal:
            path = []
            while current is not None:
                path.append(current)
                current = parent[current]
            path.reverse()
            print("Path found:", " → ".join(path))
            print("Total cost:", g_cost[goal])
            return

        open_list.remove(current)

        for neighbor, cost in graph[current].items():
            temp_g = g_cost[current] + cost

            if neighbor not in g_cost or temp_g < g_cost[neighbor]:
                g_cost[neighbor] = temp_g
                parent[neighbor] = current
                if neighbor not in open_list:
                    open_list.append(neighbor)

    print("No path found!")

A_star_search('A', 'G')
```

Lab Task:

1. Implement the A* search algorithm to find the path from State A to J. Find the Optimal Path's Order of Expansion and the Total Cost.



Lab # 05 Marks distribution

	ER1	ER6	ER8
Task	3 points	3 points	4 points

Lab # 05 Rubric Evaluation Guideline:

#	Qualities & Criteria	0 < Poor <= 1	1 < Satisfactory <= 2	2 < Excellent <=3
ER1	**Task Completion**	Minimal or no program functionality was achieved.	Some tasks were completed, but the program has errors or incomplete functionalities.	All tasks were completed, and the program runs without errors.
#	**Qualities & Criteria**	**0 < Poor <= 1**	**1 < Satisfactory <= 2**	**2 < Excellent <=3**
ER6	**Program Output**	Output is inaccurate or poorly presented.	Output is mostly accurate but may lack labels, captions, or formatting.	Output is clear, accurate, and well presented with labels, captions, and proper formatting.
#	**Qualities & Criteria**	**0 < Poor <= 1**	**1.5 < Satisfactory <= 2.5**	**2.5 < Excellent <= 4**
ER8	**Question & Answer**	Answers some questions but not confidently or based on lab task knowledge.	Answers most questions confidently and based on lab task knowledge.	Answers all questions confidently and demonstrates a deep understanding of the given lab task.